

Advanced Programming

Design Patterns



Design Patterns

1. Introduction

- What Are Design Patterns?
- Historical Context – The Gang Of Four (GoF)
- The GoF Design Patterns Catalog

2. Foundations for Design Patterns

- Interfaces for High Cohesion and Loose Coupling
- Abstract Base Classes as Interfaces
- Inheritance and Composition
- How to Read and Apply Design Patterns

Design Patterns

3. Selected Design Patterns

- Strategy
- Facade
- Adapter
- Composite
- Singleton
- Model-View-Controller (MVC)

4. Conclusion

- When to Use Design Patterns
- Outlook and Project Task

1. Introduction

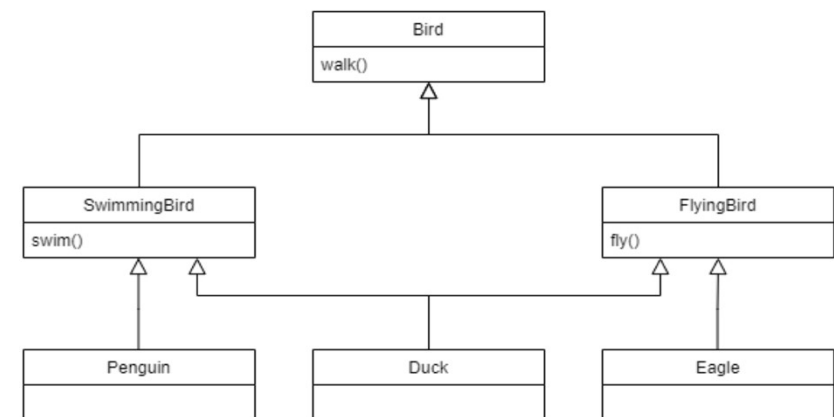
- What Are Design Patterns?
- Historical Context - The Gang Of Four (GoF)
- The GoF Design Patterns Catalog

What Are Design Patterns?

- Design patterns describe reusable collaboration structures for recurring design problems.
- A pattern names a problem, a solution structure, and the consequences of that structure.
- For example: inheritance with `is-a` relationships provides reuse, but `is-a` is not always a good solution, e.g., for multiple inheritance.

→ *How could a design pattern work that allows for code reuse without `is-a` and subclasses? And when would we use it?*

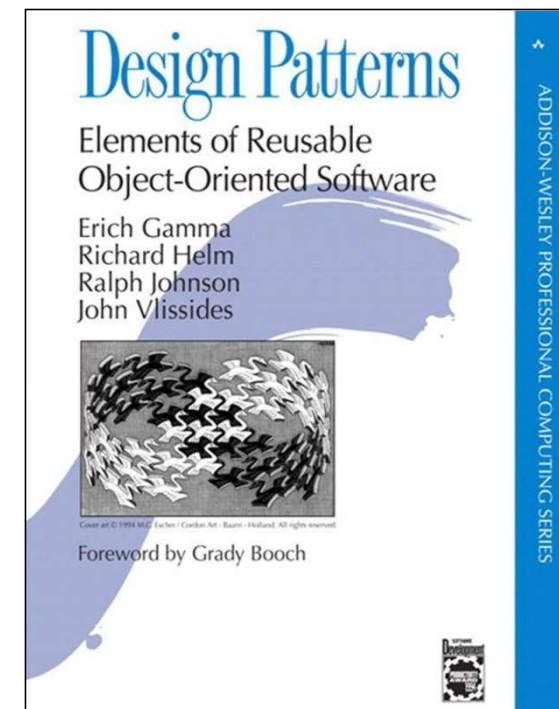
We will revisit this question in sections 2 and 3.



A design problem: how do we connect these classes? Here, multiple inheritance is used, creating a complex and fragile class hierarchy (see Chapter OOP, Part 2).

Historical Context - The Gang Of Four (GoF)

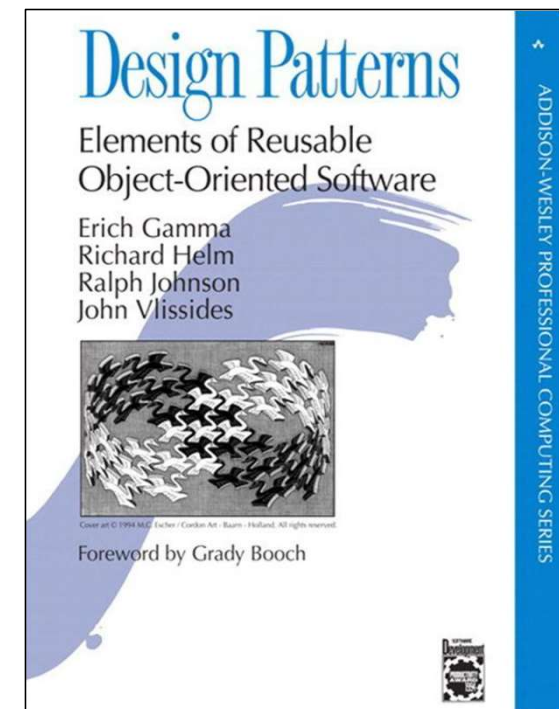
- In the late 1980s and early 1990s, object-oriented programming spread rapidly through languages such as C++ and Smalltalk.
- Teams were building larger GUI frameworks and business systems with a growing number of objects. Similar design problems and solution structures kept reappearing.
- As a consequence, design patterns were introduced by the **Gang of Four (GoF)**: Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides.
- GoF transferred the idea of a **pattern language** (Christopher Alexander 1977) to software, where reusable and maintainable design structures for large-scale architectures were needed.



Gamma et al. (1994): Design Patterns

The GoF Design Patterns Catalog

- Their 1994 book **Design Patterns: Elements of Reusable Object-Oriented Software** made pattern catalogs standard in software engineering.
- The catalog contains **23 patterns**.
- It groups them into **creational**, **structural**, and **behavioral** patterns.
- Later on, **architectural** patterns that combine multiple patterns have been suggested such as Model-View-Controller (MVC).



Gamma et al. (1994): Design Patterns

The GoF Design Patterns Catalog – Overview

Purpose		Creational	Structural	Behavioral
Scope	Class	• Factory Method	• Adapter (class)	• Interpreter • Template Method
	Object	• Abstract Factory • Builder • Prototype • Singleton	• Adapter (object) • Bridge • Composite • Decorator • Facade • Flyweight • Proxy	• Chain of Responsibility • Command • Iterator • Mediator • Memento • Observer • State • Strategy • Visitor

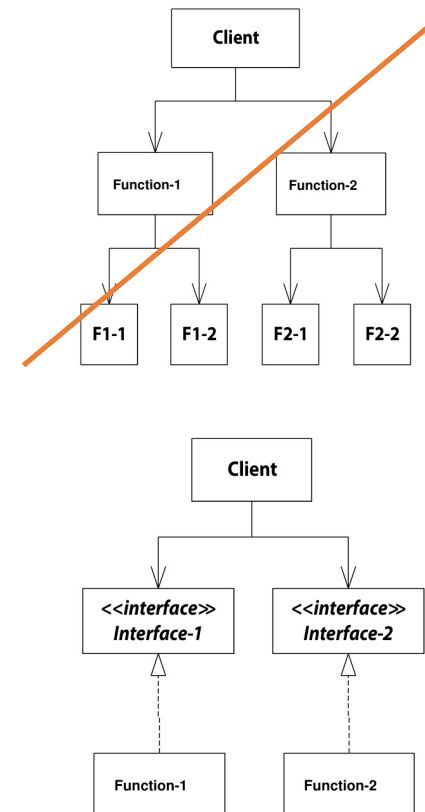
The 23 GoF design patterns in alphabetical order by purpose and scope (Gamma et al. 1994, Table 1.1). In this lecture, we focus on Strategy, Facade, Adapter, Composite, and Singleton as well the later developed MVC pattern.

2. Foundations for Design Patterns

- Interfaces for High Cohesion and Loose Coupling
- Abstract Base Classes as Interfaces
- Inheritance and Composition
- How to Read and Apply Design Patterns

Interfaces for High Cohesion and Loose Coupling

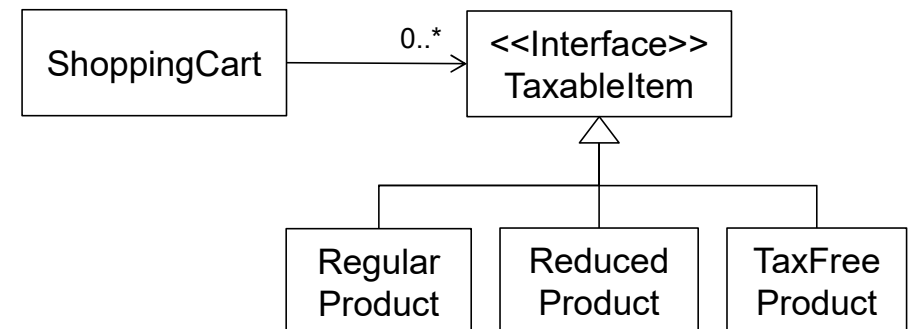
- An interface defines the operations a client may rely on without exposing the concrete class behind them.
- It separates the stable collaboration boundary from the changing implementation.
- **High cohesion and loose coupling:**
 - Using interfaces supports **loose coupling** because clients depend on a contract, not on subsystem internals.
 - They also support **high cohesion** because one service can hide its internal coordination behind one clear boundary.
- Interfaces are used in various patterns such as Strategy or Adapter
- In Python, interfaces may be realized by **Abstract Base Classes** (**ABC**, see chapter OOP, Part 2) that we previously covered



adapted from Eilebrecht and Starke (2024)

Abstract Base Classes as Interfaces

- In Python, an Abstract Base Class may be used to define an interface as a **contract**.
- The abstract class states what clients may rely on and what concrete implementations must provide.
- While the interface stays stable, concrete implementations may change.
- It declares required operations and may still hold shared state or helper behavior.
- In Python, an ABC inherits from `ABC` and required operations are marked with with `@abstractmethod`.
- The same contract can abstract **strategies** in Strategy or **services/implementors** in Bridge.



TaxableItem is an Interface we have implemented as an Abstract Base Class. A pattern emerges: The client, „ShoppingCart“, does not depend on the different implementation strategies.

adapted from OOP, Part 2

Abstract Base Classes as Interfaces – Example

```
class ShoppingCart:
    def __init__(self):
        self.items = []

    def add(self, item):
        self.items.append(item)

    def total_vat(self):
        total = 0.0

        for item in self.items:
            total = total + item.calculate_vat()

        return round(total, 2)

cart = ShoppingCart()
cart.add(RegularProduct(10.0))
cart.add(ReducedProduct(10.0))
cart.add(TaxFreeProduct(10.0))
print(cart.total_vat())
```

```
from abc import ABC, abstractmethod

class TaxableItem(ABC):
    def __init__(self, net_price):
        self.net_price = net_price

    @abstractmethod
    def calculate_vat(self):
        pass

class RegularProduct(TaxableItem):
    VAT_RATE = 0.081

    def calculate_vat(self):
        return round(self.net_price * self.VAT_RATE, 2)

class ReducedProduct(TaxableItem):
    VAT_RATE = 0.026

    def calculate_vat(self):
        return round(self.net_price * self.VAT_RATE, 2)

class TaxFreeProduct(TaxableItem):
    def calculate_vat(self):
        return 0.0
```

adapted from OOP, Part 2

Abstract Base Classes as Interfaces – LSP

- **An ABC defines the required interface:** Client code can work against `TaxableItem` and call `calculate_vat()` without checking the concrete subtype.
- The **Liskov Substitution Principle (LSP)** requires preserving the behavioral contract behind that **interface**: every subclass is compatible in its behavior with the superclass
- This hierarchy can satisfy the LSP because every subclass still fulfills the same promise: `calculate_vat()` returns the VAT for one item.

Inheritance and Composition

Aspect	Inheritance	Composition (with Delegation)
Relationship	<i>is-a</i>	<i>has-a</i>
How reuse works	Inherit behavior	Delegate work to another object
Coupling	Tight	Loose
Flexibility	Low	High
Change impact	Risky (affects subclasses)	Safer (localized changes)
Structure	Hierarchies	Modular objects
Example	Dog is an Animal	Car has an Engine (Car delegates start to Engine)
When to use	True "is-a" Use inheritance when: "This thing is fundamentally a specialized version of that thing." You're saying: • Same core identity • Same interface • Can be used interchangeably	Preferred in most cases Use composition when: "This thing is built from other things." You're saying: • Not the same identity • Just uses behavior from another object

Inheritance and Composition

- Inheritance with `is-a` relationships between classes can be used when a subclass is a true specialized form of the parent.
- Potential design problems of inheritance:
 - "The trap" (Chapter OOP, Part 2):
It might *not really* be an `is-a` relationship, e.g., `Square` and `Rectangle`.
 - Multiple inheritance can combine multiple types in one subclass, however, it may introduce *a complex and fragile class hierarchy*
- Design patterns for **composition** can be used instead of inheritance and multiple inheritance.

```
class Rectangle:
    @width.setter
    def width(self, w):
        self._width = w

    @height.setter
    def height(self, h):
        self._height = h
```

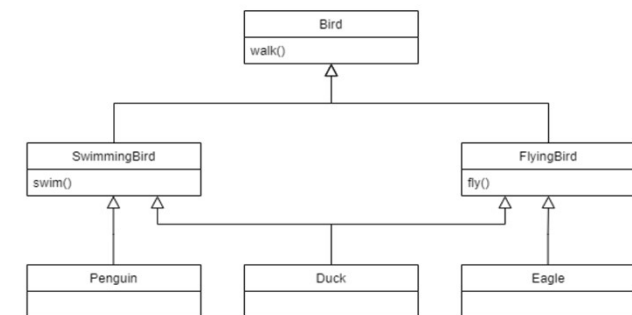


```
class Square(Rectangle):
    @width.setter
    def width(self, w):
        self._width = w
        self._height = w

    @height.setter
    def height(self, h):
        self._width = h
        self._height = h
```

Not a true `is-a` relationship

OOP, Part 2



Multiple inheritance

OOP, Part 2

Inheritance and Composition - Rules

- Rules of Good Inheritance (Recap)
 1. Subclasses must behave like their parent.
 2. Use inheritance only for true `is-a` relationships.
 3. Prefer a composition of classes with normal association relationships when functionality is re-used without `is-a` relationships.
- **Generally, *favor composition over inheritance*** and use inheritance only if `is-a` applies. That is, if *the behavior of subclasses* can replace and is compatible to *the behavior of the superclass*.
- The **Strategy** design pattern can be used for **composition**. It provides flexibility in case the behavior differs, e.g., different algorithms are needed.

How to Read and Apply Design Patterns

- Each pattern is introduced as follows
 - Type, purpose, Problem/Context, solution, advantages, disadvantages and references.
 - A diagram show the pattern first in the general case, then a Python example follows.

- **Use the purpose and Problem/Context** to determine **if the pattern applies to your project** and **where it applies**. Consider **the solution** (the pattern) and **the tradeoffs** of the advantages and disadvantages.

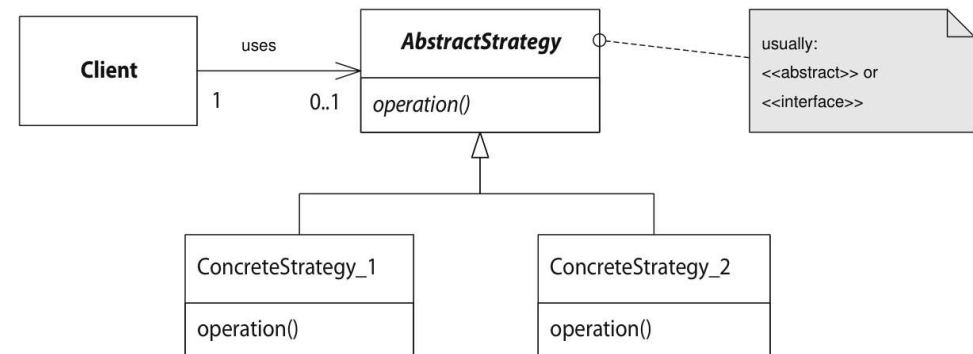
- **The goal**: clear design, code re-use / *Don't Repeat Yourself (DRO)*, better maintainability and extensibility, minization of dependencies and general flexibility.
- **However Keep It Simple (KIS(S))**: consider the tradeoffs and prefer simple over complex solutions.

3. Selected Design Patterns

- Strategy
- Facade
- Adapter
- Composite
- Singleton
- Model-View-Controller (MVC)

Strategy

- **Type:** Behavioral Pattern
- **Purpose:** vary an algorithm independently from the client and *favor composition over inheritance*.
- **Problem/Context:** the variable behavior does not justify a new subclass hierarchy or multiple inheritance.
- **Solution:** move the changing algorithm into separate strategy objects and let the context delegate through composition.

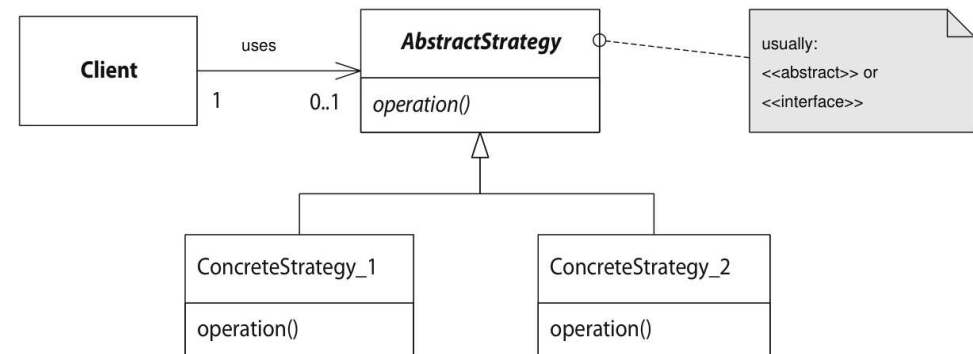


Strategy replaces concrete subclass operations with an abstract strategy that can be used by a client independent of the subclasses.

adapted from Eilebrecht and Starke (2024)

Strategy

- **Advantages:** runtime flexibility; avoids brittle inheritance trees; simpler tests; client depends on an abstraction.
- **Disadvantages:** more indirection; clients still need a rule for choosing a strategy.
- **References:** a broader variant of the pattern is the Policy pattern for business rules, consider the Adapter as an alternative



Strategy replaces concrete subclass operations with an abstract strategy that can be used by a client independent of the subclasses.

adapted from Eilebrecht and Starke (2024)

Strategy – Python Example

Client

```
class PricingService:
    def __init__(self, strategy):
        self.strategy = strategy

    def set_strategy(self, strategy):
        self.strategy = strategy

    def totals_from_lines(self, line_totals):
        subtotal = round(sum(line_totals), 2)
        discount = self.strategy.discount(subtotal)
        total = round(subtotal - discount, 2)
        return subtotal, discount, total

lines = [18.0, 22.0, 19.0]
pricing = PricingService(NoDiscount())
print("regular", pricing.totals_from_lines(lines))
pricing.set_strategy(PercentageDiscount())
print("percentage", pricing.totals_from_lines(lines))
pricing.set_strategy(FlatDiscount())
print("flat", pricing.totals_from_lines(lines))
print("percentage", pricing.totals_from_lines(lines))
```

```
from abc import ABC, abstractmethod
```

```
class DiscountStrategy(ABC):
    @abstractmethod
    def discount(self, subtotal):
        pass
```

```
class NoDiscount(DiscountStrategy):
    def discount(self, subtotal):
        return 0.0
```

```
class PercentageDiscount(DiscountStrategy):
    def discount(self, subtotal):
        if subtotal > 50.0:
            return round(subtotal * 0.10, 2)
        return 0.0
```

```
class FlatDiscount(DiscountStrategy):
    def discount(self, subtotal):
        if subtotal > 50.0:
            return 5.0
        return 0.0
```

Abstract
Strategy

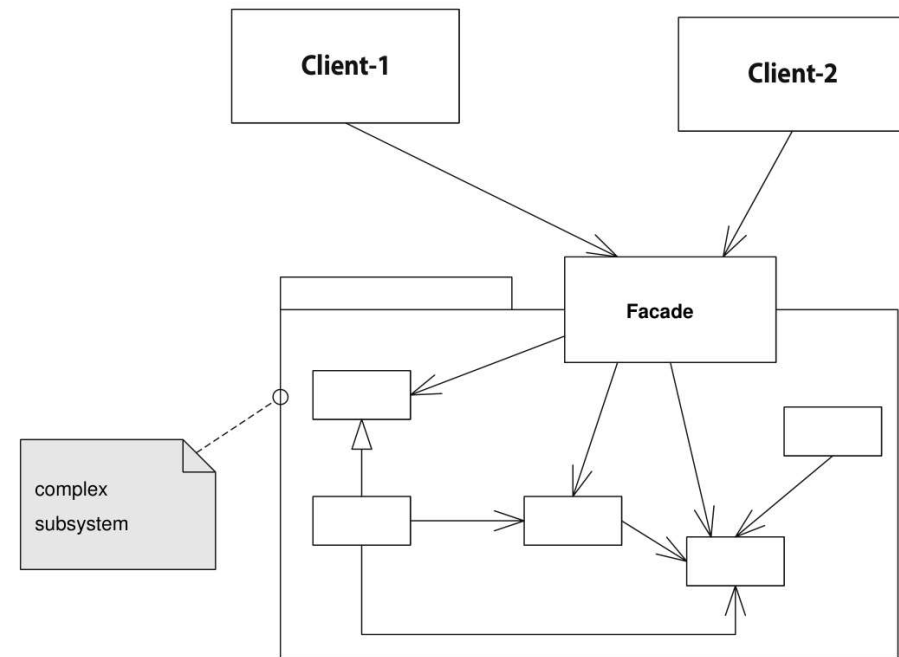
Concrete
Strategy 1

Concrete
Strategy 2

Concrete
Strategy 3

Facade

- **Type:** Structural Pattern
- **Purpose:** A **Facade** is an interface to a subsystem. It hides a complex subsystem behind one interface or ABC.
- **Problem/Context:** clients need a stable interface, not dependencies to classes in the subsystem or detailed subsystem knowledge.
- **Solution:**
 - A facade coordinates the relevant subsystem objects behind one coarse-grained API.
 - It offers one simplified entry point to a subsystem.
 - Clients talk to the facade instead of coordinating many subsystem classes directly.

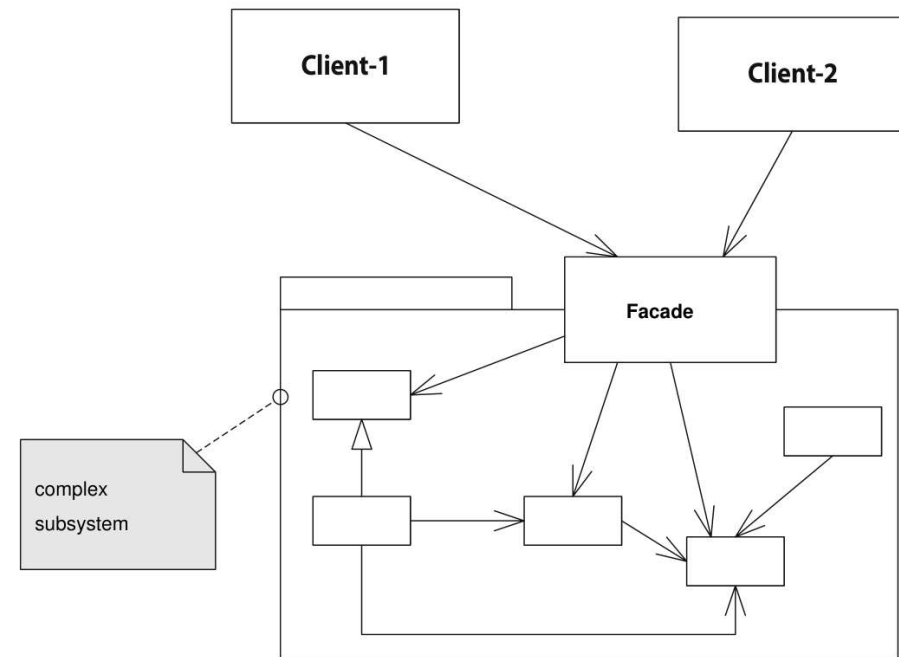


Clients see one coarse-grained entry point instead of many collaborating objects.

adapted from Eilebrecht and Starke (2024)

Facade

- **Advantages:** lower coupling, fewer integration points, simpler client code.
 - Improvement of **loose coupling** because clients depend on one stable boundary instead of many internal details.
 - Improvement of **cohesion** because one dedicated object hides and manages the *orchestration* of all objects in the subsystem.
- **Disadvantages:** the facade may absorb too many responsibilities.
- **References:** variants exist, e.g., for remote and service facades; related to Adapter and Proxy



Clients see one coarse-grained entry point instead of many collaborating objects.

adapted from Eilebrecht and Starke (2024)

Facade – Python Example

Client 1

```
class WebShopClient:
    def __init__(self, facade):
        self.facade = facade

    def checkout(self):
        self.facade.place_order("Alice", "PIZZA-MARGHERITA")
```

Client 2

```
class SupportDeskClient:
    def __init__(self, facade):
        self.facade = facade

    def reorder(self):
        self.facade.place_order("Bob", "PIZZA-MARGHERITA")
```

```
facade = OrderFacade()
WebShopClient(facade).checkout()
SupportDeskClient(facade).reorder()
```

Facade

```
class OrderFacade:
    def __init__(self):
        self.inventory = InventoryService()
        self.payments = PaymentService()
        self.delivery = DeliveryService()

    def place_order(self, customer, sku):
        self.inventory.reserve(sku)
        self.payments.charge(customer)
        self.delivery.schedule(customer, sku)
```

Subsystem

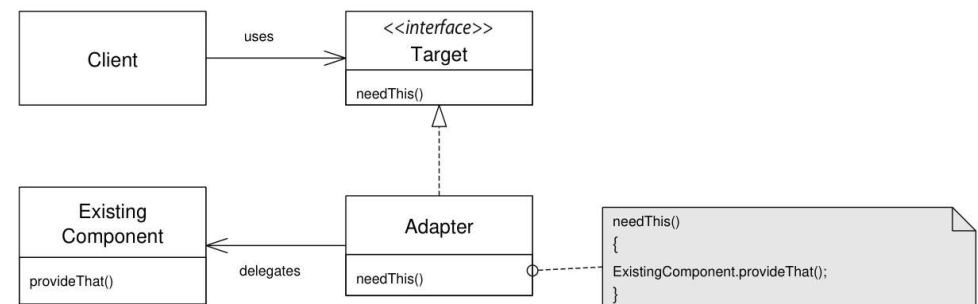
```
class InventoryService:
    def reserve(self, sku):
        print(f"reserve {sku}")

class PaymentService:
    def charge(self, customer):
        print(f"charge {customer}")

class DeliveryService:
    def schedule(self, customer, sku):
        print(f"deliver {sku} to {customer}")
```

Adapter

- **Type:** Structural Pattern
- **Purpose:** make an existing component usable through the interface the client expects.
- **Problem/Context:** an external or legacy component has useful behavior but the wrong API.
- **Solution:** expose the target interface and translate calls to the adaptee.

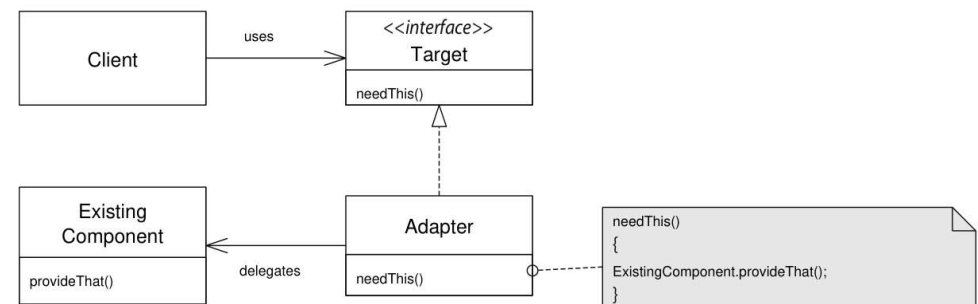


The Adapter translates a call from Client to Target without changing the Existing Component.

adapted from Eilebrecht and Starke (2024)

Adapter

- **Advantages:** reuse existing components, isolate external API changes.
- **Disadvantages:** translation logic adds indirection and can hide expensive work.
- **References:** related to Facade, Proxy, Wrapper, and Gateway.



The Adapter translates a call from Client to Target without changing the Existing Component.

adapted from Eilebrecht and Starke (2024)

Adapter – Python Example

Client

```
class CheckoutService:
    def __init__(self, payment_provider):
        self.payment_provider = payment_provider

    def place_order(self, amount_chf):
        self.payment_provider.pay(amount_chf)
        print("order confirmed")

gateway = PaymentGateway()
adapter = PaymentAdapter(gateway)
checkout = CheckoutService(adapter)
checkout.place_order(19.50)
```

Existing
Component

```
class PaymentGateway:
    def charge_cents(self, cents):
        print(f"charging {cents} cents")
```

```
from abc import ABC, abstractmethod
```

```
class PaymentProvider(ABC):
    @abstractmethod
    def pay(self, amount_chf):
        pass
```

```
class PaymentAdapter(PaymentProvider):
    def __init__(self, gateway):
        self.gateway = gateway

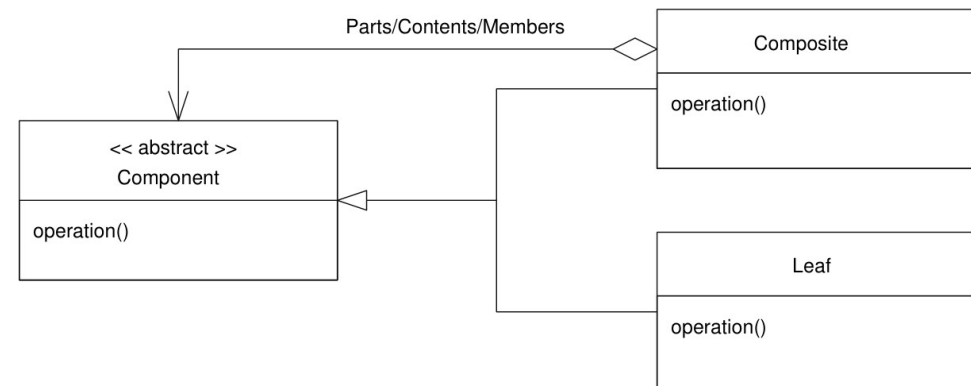
    def pay(self, amount_chf):
        cents = int(amount_chf * 100)
        self.gateway.charge_cents(cents)
```

Target
Interface

Adapter

Composite

- **Type:** Structural Pattern
- **Purpose:** treat individual elements and nested groups uniformly, e.g., for nested folders with files and further folders, for other tree structures, or nested UI components.
- **Problem/Context:** the client works with a tree structure. It should not make a difference to access any part of the tree, the whole tree (e.g. all files), a part of a tree (e.g. a subdirectory), or a leaf (a file).
- **Solution:** leaf and composite implement the same component operation.

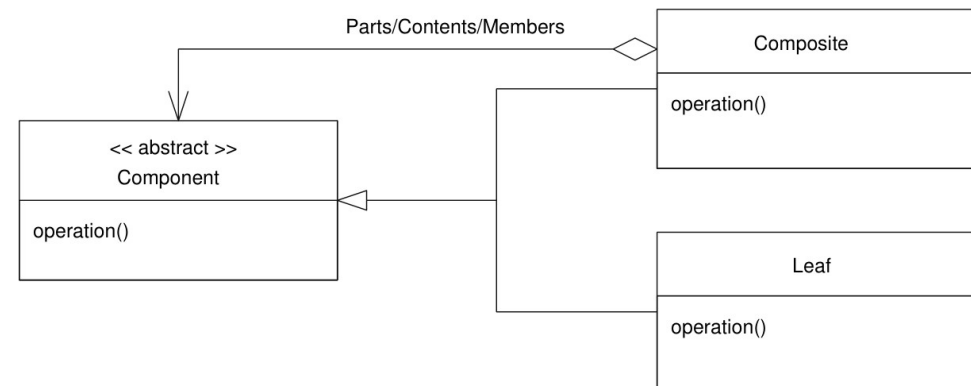


Client code can call one operation on a leaf or on a whole tree.

adapted from Eilebrecht and Starke (2024)

Composite

- **Advantages:** uniform client code, regardless of where the tree is accessed.
- **Disadvantages:** the component is a shared interface for all composite elements, e.g., several files. Therefore, the composite can become over-complicated.
- **References:** related to Iterator



Client code can call one operation on a leaf or on a whole tree.

adapted from Eilebrecht and Starke (2024)

Composite – Python Example

Component

```
from abc import ABC, abstractmethod

class Node(ABC):
    @abstractmethod
    def total_size(self):
        pass
```

Composite

```
class Folder(Node):
    def __init__(self, name):
        self.name = name
        self.children = []

    def add(self, child):
        self.children.append(child)

    def total_size(self):
        return sum(child.total_size() for child in self.children)

class File(Node):
    def __init__(self, name, size):
        self.name = name
        self.size = size

    def total_size(self):
        return self.size

root = Folder("docs")
root.add(File("readme.txt", 4))
images = Folder("images")
images.add(File("logo.png", 20))
root.add(images)
print(root.total_size())
```

Leaf

Singleton

- **Type:** Creational Pattern
- **Purpose:** ensure that only one shared instance of a class is created. It acts as one well-defined access point.
- **Problem/Context:** the application needs one global coordination object, multiple objects should not exist.
- **Solution:** in the constructor, restrict construction such that only one object can be created and return the same instance on repeated access.

SingletonClass
– _instance : SingletonClass
+ get_instance() : SingletonClass

adapted from Eilebrecht and Starke (2024)

Singleton

- **Advantages:** only one object can be created and called with a shared state by other objects
- **Disadvantages:** there is a global state that is hidden, allowing only one instance does not follow object orientation, tests need to account for the singleton.

References: related to Registry and Factory Method.

SingletonClass
– _instance : SingletonClass
+ get_instance() : SingletonClass

adapted from Eilebrecht and Starke (2024)

Singleton – Python Example

Singleton

```
class AppConfig:
    _instance = None

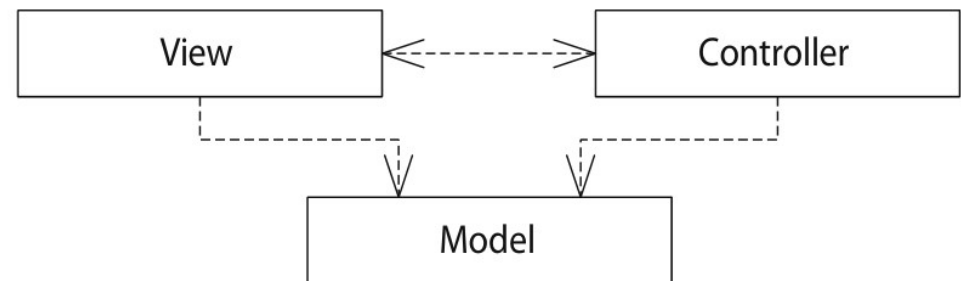
    def __init__(self):
        self.values = {"database_url": "sqlite:///app.db"}

    @classmethod
    def get_instance(cls):
        if cls._instance is None:
            cls._instance = AppConfig()
        return cls._instance

a = AppConfig.get_instance()
b = AppConfig.get_instance()
b.values["debug"] = True
print(a is b, a.values)
```

Model-View-Controller (MVC)

- **Type:** Architectural pattern.
- **Purpose:** separate presentation, application state, and input handling.
- **Problem/Context:** separation of UI, control flow and data model. E.g., the UI should not depend on the application logic and its control flow, and it should minimize dependencies to the model.
- **Solution:** split responsibilities into **Model**, **View**, and **Controller (MVC)**.

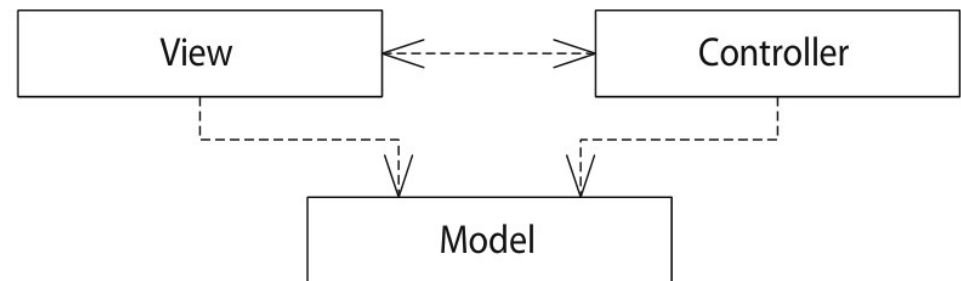


General MVC structure: View and Controller coordinate through the Model while keeping responsibilities separated.

adapted from Eilebrecht and Starke (2024)

Model-View-Controller (MVC)

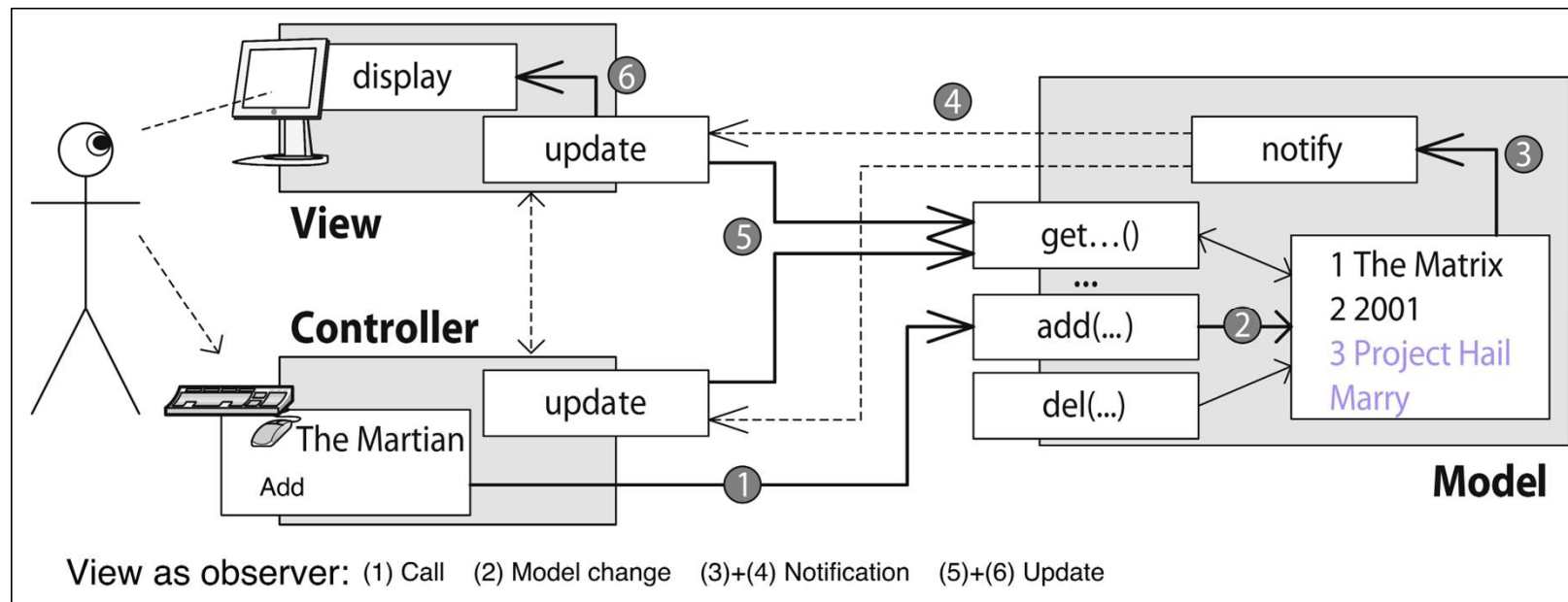
- **Advantages:** program and replace views, controllers and models largely independently, view updates are triggered by changed model data, model data can trigger view updates.
- **Disadvantages:** complex structure and coordination between classes required.
- **References:** related to Observer and Strategy.



General MVC structure: View and Controller coordinate through the Model while keeping responsibilities separated.

adapted from Eilebrecht and Starke (2024)

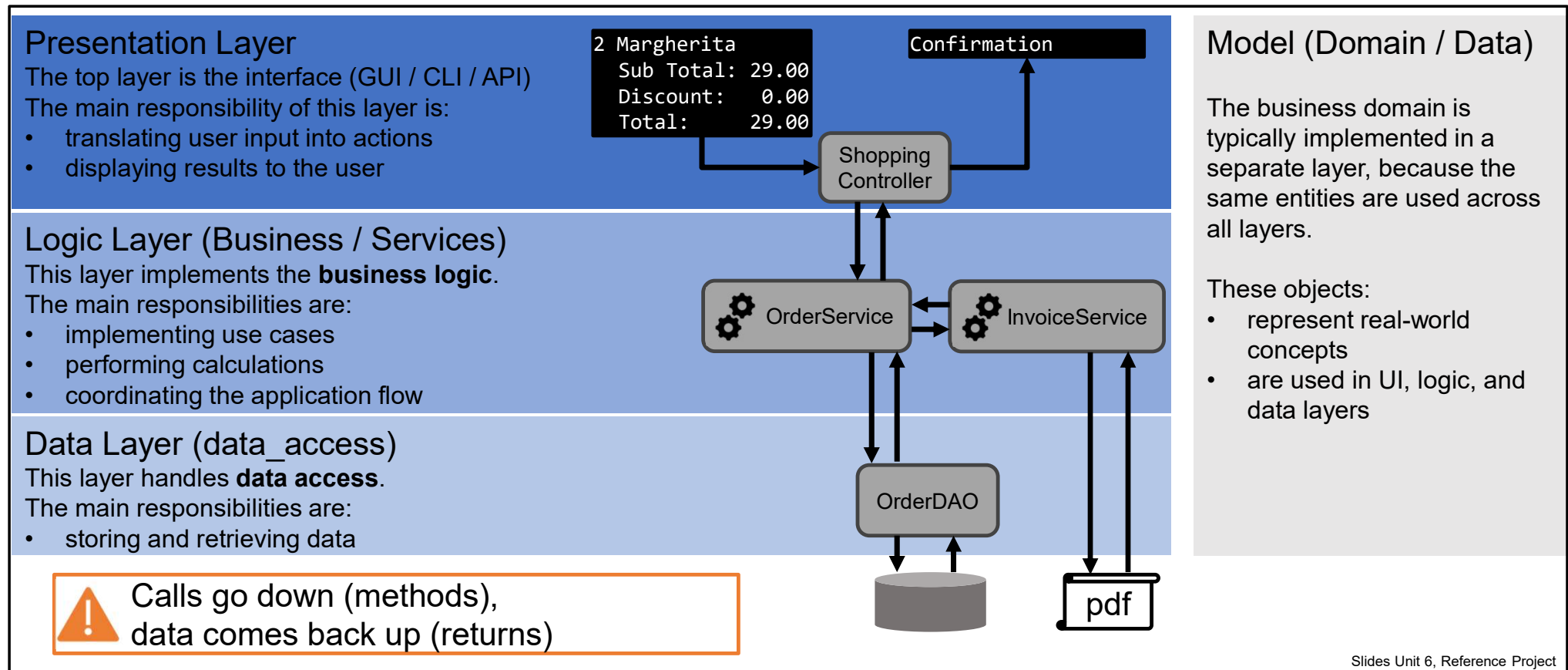
MVC – Application Example



Application example: the controller changes the model, the model notifies observers, and the view updates.

adapted from Eilebrecht and Starke (2024)

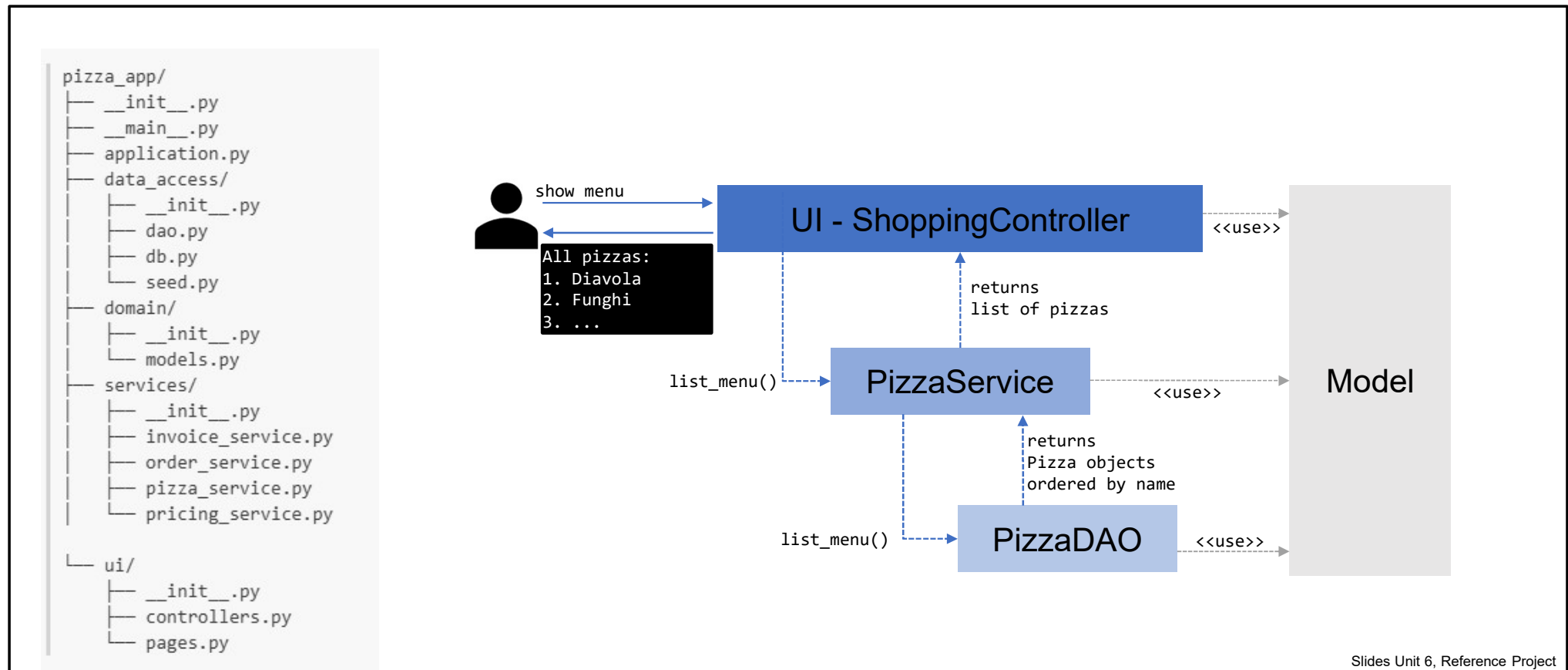
MVC in the N-Tier Architecture of the Reference Project (Recap)



Slides Unit 6, Reference Project

Reference project: MVC is part of a layered design where UI calls controllers, services coordinate use cases, and DAOs access data.

MVC in the Reference Project (Recap)



Reference project: MVC is part of a layered design where UI calls controllers, services coordinate use cases, and DAOs access data.

MVC – Python Example

```
class ShoppingController:
    def __init__(self, pizza_service):
        self.pizza_service = pizza_service

    def menu(self):
        return self.pizza_service.list_menu()

class Pages:
    def __init__(self, shopping_controller):
        self.shopping_controller = shopping_controller

    def show_menu(self):
        for pizza in self.shopping_controller.menu():
            print(pizza.name, pizza.price_chf)

pages = Pages(ShoppingController(PizzaService(PizzaDAO())))
pages.show_menu()
```

View

```
class PizzaService:
    def __init__(self, pizza_dao):
        self.pizza_dao = pizza_dao

    def list_menu(self):
        return self.pizza_dao.list_menu()
```

Controller

```
class PizzaDAO:
    def list_menu(self):
        return [
            Pizza("Diavola", 16.50),
            Pizza("Funghi", 15.00),
        ]
```

Model
(DAO)

```
class Pizza:
    def __init__(self, name, price_chf):
        self.name = name
        self.price_chf = price_chf
```

Model

minimal example adapted from the Reference Project

4. Conclusion

- When To Use Design Patterns
- Outlook and Project Task

When To Use Design Patterns

- Use a design pattern when a **design problem can be solved by it, not for the sake of applying patterns.**
- **Overengineering risk:** apply patterns for the sake of patterns may result in unnecessary abstractions, extra indirection, and more code to explain, test, and maintain.
- **Use the purpose and problem/context** to determine if the **pattern applies to your project** and **where** it applies.
- **Consider the solution** (the pattern) and the **tradeoffs** of the advantages and disadvantages.

Outlook and Project Task

- For foundational design, consider the foundations we covered:
inheritance limitations, interfaces, ABC as interfaces, and composition vs inheritance.
 - Implement **design patterns that solve problems** and offer useful trade-offs with advantages. This should be the case for
 - **Strategy** for general flexibility and maintainability
 - **Singleton** such as for database access
 - **MVC** to separate UI, control flow and model data,
and possibly also **Adapter**, **Facade**, and **Composite**.
- Task: Consider now where to apply these patterns in your project.
- **Goal:** a **clear design** with **code reuse** as in ***Don't Repeat Yourself (DRY)***, **maintainability**, **extensibility**, **minimized dependencies**, and **flexibility**.
 - ***However, Keep It Simple (KISS)***: consider the tradeoffs and prefer simple solutions over complex ones.